Dr. Ekta API - OpenAPI

Dr. Ekta Narula Dental Care API — OpenAPI (v1.1.0)

Generated from docs/openapi.yaml (machine-readable source of truth). Render with Redoc/Swagger UI for interactive docs.

`yaml openapi: 3.0.3 info: title: Dr. Ekta Narula Dental Care API version: 1.1.0 description: > Backend API (current code, 2FA removed; review update 2026-09-21: audit logging removed, consent gate removed, content is draft/published only with direct status writes). Public prefix /api/public, admin prefix /api/admin. See docs/BACKEND_DOCUMENTATION.md for the handover guide, gotchas and known bugs. servers: - url: http://localhost:4000 description: Local dev security: [] tags: - name: Root - name: Public Content - name: Enquiry - name: Booking - name: SEO - name: Admin Auth - name: Admin Clinic - name: Admin Media - name: Admin Content - name: Admin Leads - name: Admin Misc components: securitySchemes: bearerAuth: type: http scheme: bearer bearerFormat: JWT schemas: Pagination: type: object properties: page: { type: integer, example: 1 } limit: { type: integer, example: 20 } total: { type: integer, example: 5 } pages: { type: integer, example: 1 } PagedResponse: type: object properties: items: { type: array, items: { type: object } } pagination: { $ref: '#/components/schemas/Pagination' } Error: type: object properties: error: { type: string, example: Validation failed } code: { type: string, example: VALIDATION_ERROR } Treatment: type: object properties: _id: { type: string } slug: { type: string, example: root-canal } title: { type: string } status: { type: string, enum: [draft, published] } locationId: { type: string, nullable: true } sections: type: object properties: what: { type: string } who: { type: string } procedure: { type: string } pain: { type: string } recovery: { type: string } risks: { type: string } faqs: type: array items: type: object properties: question: { type: string } answer: { type: string } order: { type: integer } heroImage: { type: object, nullable: true } gallery: { type: array, items: { type: object } } relatedTreatments: { type: array, items: { type: object } } seo: type: object properties: metaTitle: { type: string } metaDescription: { type: string } ogImage: { type: object, nullable: true } createdBy: { type: object, nullable: true, description: Excluded from public payloads } publishedAt: { type: string, format: date-time, nullable: true } createdAt: { type: string, format: date-time } updatedAt: { type: string, format: date-time } Article: type: object properties: _id: { type: string } title: { type: string } slug: { type: string } category: type: string enum: [preventive-care, restorative, cosmetic, orthodontics, oral-surgery, pediatric, emergency, technology, patient-education] tags: { type: array, items: { type: string } } body: { type: string, description: Omitted from list endpoint } author: { type: string } reviewer: { type: object, nullable: true } credentialsDisplay: { type: string } sources: type: array items: type: object properties: title: { type: string } url: { type: string } authors: { type: string } publishedDate: { type: string, format: date-time } status: { type: string, enum: [draft, published] } locationId: { type: string, nullable: true } publishedAt: { type: string, format:

date-time, nullable: true } seo: { type: object } createdAt: { type: string, format: date-time } updatedAt: { type: string, format: date-time } Testimonial: type: object properties: _id: { type: string } patientDisplayName: { type: string } treatmentTag: { type: object, nullable: true } type: { type: string, enum: [text, video, audio] } content: { type: string } status: { type: string, enum: [draft, published] } locationId: { type: string, nullable: true } publishedAt: { type: string, format: date-time, nullable: true } createdAt: { type: string, format: date-time } updatedAt: { type: string, format: date-time } GalleryItem: type: object properties: _id: { type: string } beforeImg: { type: object } afterImg: { type: object } treatmentTag: { type: object, nullable: true } content: { type: string } status: { type: string, enum: [draft, published] } locationId: { type: string, nullable: true } publishedAt: { type: string, format: date-time, nullable: true } createdAt: { type: string, format: date-time } updatedAt: { type: string, format: date-time } Video: type: object properties: _id: { type: string } url: { type: string } title: { type: string } category: { type: string } thumbnail: { type: string } order: { type: integer } status: { type: string, enum: [published, hidden] } locationId: { type: string, nullable: true } transcript: { type: string } captionsUrl: { type: object, nullable: true } createdAt: { type: string, format: date-time } updatedAt: { type: string, format: date-time } CuratedPost: type: object properties: _id: { type: string } image: { type: object } caption: { type: string } permalink: { type: string } order: { type: integer } locationId: { type: string, nullable: true } createdAt: { type: string, format: date-time } updatedAt: { type: string, format: date-time } Announcement: type: object properties: _id: { type: string } title: { type: string } body: { type: string } linkUrl: { type: string } status: { type: string, enum: [draft, published] } order: { type: integer } locationId: { type: string, nullable: true } publishedAt: { type: string, format: date-time, nullable: true } createdAt: { type: string, format: date-time } updatedAt: { type: string, format: date-time } Lead: type: object properties: _id: { type: string } name: { type: string } phone: { type: string } treatmentInterest: { type: object, nullable: true } source: { type: string, enum: [enquiry_form, practo_click, phone_click] } locationId: { type: string, nullable: true } date: { type: string, format: date-time } leadStatus: { type: string, enum: [new, contacted, scheduled, completed, lost] } appointmentStatus: { type: string, enum: [none, booked, completed, cancelled, no_show] } notes: { type: string } createdAt: { type: string, format: date-time } updatedAt: { type: string, format: date-time } MediaAsset: type: object properties: _id: { type: string } filename: { type: string } originalName: { type: string } storageKey: { type: string } url: { type: string, description: Original delivery URL } optimizedUrl: { type: string, nullable: true, description: f_auto,q_auto delivery URL (images) } variants: type: array items: type: object properties: name: { type: string, enum: [thumb, medium, large] } width: { type: integer } url: { type: string } mimeType: { type: string } size: { type: integer } uploadedBy: { type: object } usedIn: { type: array, items: { type: object } } altText: { type: string } createdAt: { type: string, format: date-time } updatedAt: { type: string, format: date-time } ClinicInfo: type: object properties: _id: { type: string } practiceName: { type: string } address: type: object properties: street: { type: string } city: { type: string } state: { type: string } pincode: { type: string } country: { type: string } phones: { type: array, items: { type: string } } timings: type: array items: type: object properties: day: { type: string } open: { type: string } close: { type: string } isClosed: { type: boolean } mapCoordinates: type: object properties: latitude: { type: number } longitude: { type: number } socialLinks: type: array items: type: object properties: platform: { type: string } url: { type: string } legalEntityName: { type: string } email: { type: string } locationId: { type: string, nullable: true } createdAt: { type: string, format: date-time } updatedAt: { type: string, format: date-time } parameters: PageParam: name: page in: query schema: { type: integer, minimum: 1, default: 1 } LimitParam: name: limit in: query schema: { type: integer, minimum: 1, maximum: 100, default: 20 } IdParam: name: id in: path required: true schema: { type: string, description: MongoDB ObjectId } SlugParam: name: slug in:

path required: true schema: { type: string } paths: /: get: tags: [Root] summary: API info responses: ‘200’: { description: OK, content: { application/json: { schema: { type: object } } } } }

————- Public ————- /api/public/health: get: tags: [Public Content] summary: Health check responses: ‘200’: { description: OK, content: { application/json: { schema: { type: object } } } } } /api/public/clinic-info: get: tags: [Public Content] summary: Public clinic info (cached 1h) responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/ClinicInfo’ } } } } } /api/public/treatments: get: tags: [Public Content] summary: List published treatments parameters: - $ref: ‘#/components/parameters/PageParam’ - $ref: ‘#/components/parameters/LimitParam’ - { name: search, in: query, schema: { type: string } } responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/PagedResponse’ } } } } } /api/public/treatments/{slug}: get: tags: [Public Content] summary: Published treatment by slug parameters: - $ref: ‘#/components/parameters/SlugParam’ responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/Treatment’ } } } } ‘404’: { description: Not found, content: { application/json: { schema: { $ref: ‘#/components/schemas/Error’ } } } } } /api/public/articles: get: tags: [Public Content] summary: List published articles (body omitted) parameters: - $ref: ‘#/components/parameters/PageParam’ - $ref: ‘#/components/parameters/LimitParam’ - { name: category, in: query, schema: { type: string } } - { name: tags, in: query, schema: { type: array, items: { type: string } } } - { name: search, in: query, schema: { type: string } } responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/PagedResponse’ } } } } /api/public/articles/{slug}: get: tags: [Public Content] summary: ‘Published article by slug (full body). Unknown slug: 404’ parameters: - $ref: ‘#/components/parameters/SlugParam’ responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/Article’ } } } } } /api/public/profile/: get: tags: [Public Content] summary: Doctor profile singleton responses: ‘200’: { description: OK, content: { application/json: { schema: { type: object } } } } } put: tags: [Admin Misc] summary: Update profile (requires token despite public prefix) security: [{ bearerAuth: [] }] requestBody: { content: { application/json: { schema: { type: object } } } } responses: ‘200’: { description: OK, content: { application/json: { schema: { type: object } } } } ‘401’: { description: Unauthorized } ‘422’: { description: Validation failed } /api/public/testimonials/: get: tags: [Public Content] summary: List published testimonials parameters: - $ref: ‘#/components/parameters/PageParam’ - $ref: ‘#/components/parameters/LimitParam’ - { name: treatmentTag, in: query, schema: { type: string } } responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/PagedResponse’ } } } } /api/public/gallery/: get: tags: [Public Content] summary: List published gallery items parameters: - $ref: ‘#/components/parameters/PageParam’ - $ref: ‘#/components/parameters/LimitParam’ - { name: treatmentTag, in: query, schema: { type: string } } responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/PagedResponse’ } } } } /api/public/videos/: get: tags: [Public Content] summary: List published videos parameters: - $ref: ‘#/components/parameters/PageParam’ - $ref: ‘#/components/parameters/LimitParam’ - { name: category, in: query, schema: { type: string } } responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/PagedResponse’ } } } } /api/public/instagram-posts/: get: tags: [Public Content] summary: List curated Instagram posts parameters: - $ref: ‘#/components/parameters/PageParam’ - $ref: ‘#/components/parameters/LimitParam’ responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/PagedResponse’ } } } } /api/public/announcements/: get: tags: [Public Content] summary: List published

announcements (ordered) parameters: - $ref: '#/components/parameters/PageParam' - $ref: '#/components/parameters/LimitParam' responses: '200': { description: OK, content: { application/json: { schema: { $ref: '#/components/schemas/PagedResponse' } } } } /api/public/home/: get: tags: [Public Content] summary: Homepage aggregate (treatments/articles/videos/testimonials) responses: '200': { description: OK, content: { application/json: { schema: { type: object } } } } /api/public/home/config: get: tags: [Admin Misc] summary: Homepage featured config (requires token despite public prefix) security: [{ bearerAuth: [] }] responses: '200': { description: OK, content: { application/json: { schema: { type: object } } } } '401': { description: Unauthorized } put: tags: [Admin Misc] summary: Update homepage featured config (requires token) security: [{ bearerAuth: [] }] requestBody: { content: { application/json: { schema: { type: object } } } } responses: '200': { description: OK, content: { application/json: { schema: { type: object } } } } '401': { description: Unauthorized } /api/public/enquiry: post: tags: [Enquiry] summary: Submit patient enquiry (creates lead) requestBody: required: true content: application/json: schema: type: object required: [name, phone] properties: name: { type: string, maxLength: 100 } phone: { type: string, example: '+91-9876543210' } treatmentInterest: { type: string, description: Treatment ObjectId } honeypot: { type: string, description: Must stay empty } captchaToken: { type: string, description: Verified server-side when RECAPTCHA_SECRET is set } responses: '201': { description: Created, content: { application/json: { schema: { type: object } } } } '422': { description: Validation failed } '429': { description: Rate limited } /api/public/booking-cta: get: tags: [Booking] summary: Active Practo booking links responses: '200': { description: OK, content: { application/json: { schema: { type: object } } } } '404': { description: No active provider } /api/public/leads/practo-click: post: tags: [Booking] summary: Log Practo click (always 200) responses: '200': { description: OK } /api/public/sitemap.xml: get: tags: [SEO] summary: Generated sitemap XML responses: '200': { description: OK, content: { application/xml: { schema: { type: string } } } } /api/public/robots.txt: get: tags: [SEO] summary: Generated robots.txt responses: '200': { description: OK, content: { text/plain: { schema: { type: string } } } } /api/public/redirect/{path}: get: tags: [SEO] summary: Resolve legacy URL redirect (301/302) parameters: - { name: path, in: path, required: true, schema: { type: string } } responses: '301': { description: Moved permanently } '302': { description: Found } '404': { description: No redirect configured }

———— Admin auth ———— /api/admin/auth/login: post: tags: [Admin Auth] summary: Admin login (email + password) requestBody: required: true content: application/json: schema: type: object required: [email, password] properties: email: { type: string, format: email } password: { type: string } responses: '200': { description: OK, content: { application/json: { schema: { type: object } } } } '401': { description: Invalid credentials / disabled } '422': { description: Validation failed } '429': { description: Locked / rate limited } /api/admin/auth/refresh: post: tags: [Admin Auth] summary: Rotate token pair requestBody: required: true content: application/json: schema: type: object required: [refreshToken] properties: refreshToken: { type: string } responses: '200': { description: OK, content: { application/json: { schema: { type: object } } } } '401': { description: Invalid/revoked token } /api/admin/auth/logout: post: tags: [Admin Auth] summary: Revoke refresh token requestBody: content: application/json: schema: type: object properties: refreshToken: { type: string } responses: '200': { description: OK } /api/admin/auth/me: get: tags: [Admin Auth] summary: Current admin (requires token) responses: '200': { description: OK } '500': { description: Broken without middleware }

———— Admin clinic / media ———— /api/admin/clinic-info: get: tags: [Admin Clinic] summary: Clinic info (admin, no cache) security: [{ bearerAuth: [] }] responses: '200': { description: OK, content: { application/json: { schema: { $ref:

‘#/components/schemas/ClinicInfo’ } } } } ‘401’: { description: Unauthorized } put: tags: [Admin Clinic] summary: Update clinic info security: [{ bearerAuth: [] }] requestBody: { content: { application/json: { schema: { type: object } } } } responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/ClinicInfo’ } } } } ‘401’: { description: Unauthorized } ‘422’: { description: Validation failed } /api/admin/media/upload: post: tags: [Admin Media] summary: Get Cloudinary signed upload params security: [{ bearerAuth: [] }] requestBody: required: true content: application/json: schema: type: object required: [filename, mimeType, size] properties: filename: { type: string } mimeType: { type: string } size: { type: integer } responses: ‘200’: { description: OK } ‘422’: { description: Type/size not allowed } /api/admin/media/confirm: post: tags: [Admin Media] summary: Confirm upload, create MediaAsset (send BOTH key and publicId; optimized variants stored automatically) security: [{ bearerAuth: [] }] requestBody: required: true content: application/json: schema: type: object required: [key, url, filename, mimeType, size] properties: key: { type: string } publicId: { type: string } url: { type: string } filename: { type: string } mimeType: { type: string } size: { type: integer } responses: ‘201’: { description: Created, content: { application/json: { schema: { $ref: ‘#/components/schemas/MediaAsset’ } } } } ‘400’: { description: Missing fields } /api/admin/media: get: tags: [Admin Media] summary: List own uploads security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/PageParam’ - $ref: ‘#/components/parameters/LimitParam’ - { name: mimeType, in: query, schema: { type: string } } responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/PagedResponse’ } } } } /api/admin/media/{id}: get: tags: [Admin Media] summary: One media asset security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/MediaAsset’ } } } } ‘404’: { description: Not found } delete: tags: [Admin Media] summary: Delete media (in-use guard never fires — see docs) security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ responses: ‘200’: { description: OK } ‘404’: { description: Not found }

———- Admin content (draft/published, direct status) ———- /api/admin/treatments: get: tags: [Admin Content] summary: List treatments (all statuses) security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/PageParam’ - $ref: ‘#/components/parameters/LimitParam’ - { name: status, in: query, schema: { type: string } } - { name: search, in: query, schema: { type: string } } responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/PagedResponse’ } } } } post: tags: [Admin Content] summary: Create treatment (draft or published) security: [{ bearerAuth: [] }] requestBody: required: true content: application/json: schema: type: object required: [title] properties: title: { type: string } slug: { type: string } status: { type: string, enum: [draft, published] } sections: { type: object } faqs: { type: array, items: { type: object } } heroImage: { type: string } gallery: { type: array, items: { type: string } } relatedTreatments: { type: array, items: { type: string } } seo: { type: object } responses: ‘201’: { description: Created, content: { application/json: { schema: { $ref: ‘#/components/schemas/Treatment’ } } } } ‘422’: { description: Validation failed } /api/admin/treatments/{id}: get: tags: [Admin Content] summary: Treatment by id security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/Treatment’ } } } } ‘404’: { description: Not found } put: tags: [Admin Content] summary: ‘Update treatment (status set directly: draft or published)’ security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ requestBody: { content: { application/json: { schema: { type: object } } } } responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/Treatment’ } } } } ‘404’: {

description: Not found } delete: tags: [Admin Content] summary: Delete treatment security: [{ bearerAuth: [] }] parameters: - $ref: '#/components/parameters/IdParam' responses: '200': { description: OK } '404': { description: Not found } /api/admin/articles: get: tags: [Admin Content] summary: List articles (all statuses) security: [{ bearerAuth: [] }] parameters: - $ref: '#/components/parameters/PageParam' - $ref: '#/components/parameters/LimitParam' - { name: status, in: query, schema: { type: string } } - { name: category, in: query, schema: { type: string } } - { name: search, in: query, schema: { type: string } } responses: '200': { description: OK, content: { application/json: { schema: { $ref: '#/components/schemas/PagedResponse' } } } } post: tags: [Admin Content] summary: Create article draft security: [{ bearerAuth: [] }] requestBody: required: true content: application/json: schema: type: object required: [title, category, body, author] properties: title: { type: string } slug: { type: string } category: { type: string } tags: { type: array, items: { type: string } } body: { type: string } author: { type: string } reviewer: { type: string } credentialsDisplay: { type: string } sources: { type: array, items: { type: object } } status: { type: string } seo: { type: object } responses: '201': { description: Created, content: { application/json: { schema: { $ref: '#/components/schemas/Article' } } } } '422': { description: Validation failed } /api/admin/articles/{id}: get: tags: [Admin Content] summary: Article by id security: [{ bearerAuth: [] }] parameters: - $ref: '#/components/parameters/IdParam' responses: '200': { description: OK, content: { application/json: { schema: { $ref: '#/components/schemas/Article' } } } } '404': { description: Not found } put: tags: [Admin Content] summary: 'Update article (status set directly: draft or published)' security: [{ bearerAuth: [] }] parameters: - $ref: '#/components/parameters/IdParam' requestBody: { content: { application/json: { schema: { type: object } } } } responses: '200': { description: OK, content: { application/json: { schema: { $ref: '#/components/schemas/Article' } } } } '404': { description: Not found } delete: tags: [Admin Content] summary: Delete article security: [{ bearerAuth: [] }] parameters: - $ref: '#/components/parameters/IdParam' responses: '200': { description: OK } '404': { description: Not found }

———— Admin testimonials / gallery (draft/published, no consent) ———— /api/admin/testimonials/: get: tags: [Admin Content] summary: List testimonials (all statuses, status filter works) security: [{ bearerAuth: [] }] parameters: - $ref: '#/components/parameters/PageParam' - $ref: '#/components/parameters/LimitParam' - { name: treatmentTag, in: query, schema: { type: string } } - { name: status, in: query, schema: { type: string, enum: [draft, published] } } responses: '200': { description: OK, content: { application/json: { schema: { $ref: '#/components/schemas/PagedResponse' } } } } post: tags: [Admin Content] summary: Create testimonial (draft or published) security: [{ bearerAuth: [] }] requestBody: required: true content: application/json: schema: type: object required: [patientDisplayName, content] properties: patientDisplayName: { type: string } treatmentTag: { type: string } type: { type: string, enum: [text, video, audio] } content: { type: string } status: { type: string, enum: [draft, published] } responses: '201': { description: Created, content: { application/json: { schema: { $ref: '#/components/schemas/Testimonial' } } } } '422': { description: Validation failed } /api/admin/testimonials/{id}: get: tags: [Admin Content] summary: One testimonial security: [{ bearerAuth: [] }] parameters: - $ref: '#/components/parameters/IdParam' responses: '200': { description: OK, content: { application/json: { schema: { $ref: '#/components/schemas/Testimonial' } } } } '404': { description: Not found } put: tags: [Admin Content] summary: 'Update testimonial (status set directly: draft or published)' security: [{ bearerAuth: [] }] parameters: - $ref: '#/components/parameters/IdParam' requestBody: { content: { application/json: { schema: { type: object } } } } responses: '200': { description: OK } '404': { description: Not found } delete: tags: [Admin Content] summary: Delete testimonial security: [{ bearerAuth: [] }] parameters: - $ref: '#/components/parameters/IdParam' responses:

‘200’: { description: OK } ‘404’: { description: Not found } /api/admin/gallery/: get: tags: [Admin Content] summary: List gallery items (all statuses, status filter works) security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/PageParam’ - $ref: ‘#/components/parameters/LimitParam’ - { name: treatmentTag, in: query, schema: { type: string } } - { name: status, in: query, schema: { type: string, enum: [draft, published] } } responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/PagedResponse’ } } } } post: tags: [Admin Content] summary: Create gallery item (beforeImg + afterImg required; draft or published) security: [{ bearerAuth: [] }] requestBody: required: true content: application/json: schema: type: object required: [beforeImg, afterImg] properties: beforeImg: { type: string } afterImg: { type: string } treatmentTag: { type: string } content: { type: string } status: { type: string, enum: [draft, published] } responses: ‘201’: { description: Created, content: { application/json: { schema: { $ref: ‘#/components/schemas/GalleryItem’ } } } } ‘422’: { description: Validation failed } /api/admin/gallery/{id}: get: tags: [Admin Content] summary: One gallery item security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/GalleryItem’ } } } } ‘404’: { description: Not found } put: tags: [Admin Content] summary: ‘Update gallery item (status set directly: draft or published)’ security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ requestBody: { content: { application/json: { schema: { type: object } } } } responses: ‘200’: { description: OK } ‘404’: { description: Not found } delete: tags: [Admin Content] summary: Delete gallery item security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ responses: ‘200’: { description: OK } ‘404’: { description: Not found }

———- Admin videos / instagram / home ———- /api/admin/videos/: get: tags: [Admin Content] summary: List videos (all statuses) security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/PageParam’ - $ref: ‘#/components/parameters/LimitParam’ - { name: status, in: query, schema: { type: string } } - { name: category, in: query, schema: { type: string } } responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/PagedResponse’ } } } } post: tags: [Admin Content] summary: Create video security: [{ bearerAuth: [] }] requestBody: required: true content: application/json: schema: type: object required: [url, title, category] properties: url: { type: string } title: { type: string } category: { type: string } thumbnail: { type: string } order: { type: integer } status: { type: string, enum: [published, hidden] } transcript: { type: string } captionsUrl: { type: string } responses: ‘201’: { description: Created, content: { application/json: { schema: { $ref: ‘#/components/schemas/Video’ } } } } ‘422’: { description: Validation failed } /api/admin/videos/{id}: get: tags: [Admin Content] summary: One video security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/Video’ } } } } ‘404’: { description: Not found } put: tags: [Admin Content] summary: Update video (send full object — validator is create-shaped) security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ requestBody: { content: { application/json: { schema: { type: object } } } } responses: ‘200’: { description: OK } ‘422’: { description: Validation failed } delete: tags: [Admin Content] summary: Delete video security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ responses: ‘200’: { description: OK } ‘404’: { description: Not found } /api/admin/instagram-posts/: get: tags: [Admin Content] summary: List curated posts (admin) security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/PageParam’ - $ref: ‘#/components/parameters/LimitParam’ responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/PagedResponse’ } } } } post: tags: [Admin Content] summary: Create curated post security: [{ bearerAuth: [] }]

requestBody: required: true content: application/json: schema: type: object required: [image, permalink] properties: image: { type: string } caption: { type: string } permalink: { type: string } order: { type: integer } responses: ‘201’: { description: Created, content: { application/json: { schema: { $ref: ‘#/components/schemas/CuratedPost’ } } } } ‘422’: { description: Validation failed } /api/admin/instagram-posts/{id}: get: tags: [Admin Content] summary: One curated post security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/CuratedPost’ } } } } ‘404’: { description: Not found } put: tags: [Admin Content] summary: Update curated post (all fields optional) security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ requestBody: { content: { application/json: { schema: { type: object } } } } responses: ‘200’: { description: OK } delete: tags: [Admin Content] summary: Delete curated post security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ responses: ‘200’: { description: OK } ‘404’: { description: Not found } /api/admin/announcements/: get: tags: [Admin Content] summary: List announcements (all statuses, status filter works) security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/PageParam’ - $ref: ‘#/components/parameters/LimitParam’ - { name: status, in: query, schema: { type: string, enum: [draft, published] } } responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/PagedResponse’ } } } } post: tags: [Admin Content] summary: Create announcement (draft or published) security: [{ bearerAuth: [] }] requestBody: required: true content: application/json: schema: type: object required: [title, body] properties: title: { type: string } body: { type: string } linkUrl: { type: string } status: { type: string, enum: [draft, published] } order: { type: integer } responses: ‘201’: { description: Created, content: { application/json: { schema: { $ref: ‘#/components/schemas/Announcement’ } } } } ‘422’: { description: Validation failed } /api/admin/announcements/{id}: get: tags: [Admin Content] summary: One announcement security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/Announcement’ } } } } ‘404’: { description: Not found } put: tags: [Admin Content] summary: ‘Update announcement (status set directly: draft or published)’ security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ requestBody: { content: { application/json: { schema: { type: object } } } } responses: ‘200’: { description: OK } ‘404’: { description: Not found } delete: tags: [Admin Content] summary: Delete announcement security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ responses: ‘200’: { description: OK } ‘404’: { description: Not found } /api/admin/profile/: get: tags: [Admin Misc] summary: Doctor profile (admin) security: [{ bearerAuth: [] }] responses: ‘200’: { description: OK, content: { application/json: { schema: { type: object } } } } put: tags: [Admin Misc] summary: Update doctor profile security: [{ bearerAuth: [] }] requestBody: { content: { application/json: { schema: { type: object } } } } responses: ‘200’: { description: OK } ‘422’: { description: Validation failed } /api/admin/home/: get: tags: [Admin Misc] summary: NOTE — serves public homepage aggregate security: [{ bearerAuth: [] }] responses: ‘200’: { description: OK } /api/admin/home/config: get: tags: [Admin Misc] summary: Homepage featured config security: [{ bearerAuth: [] }] responses: ‘200’: { description: OK } ‘401’: { description: Unauthorized } put: tags: [Admin Misc] summary: Update featured id arrays security: [{ bearerAuth: [] }] requestBody: content: application/json: schema: type: object properties: featuredTreatments: { type: array, items: { type: string } } featuredArticles: { type: array, items: { type: string } } featuredVideos: { type: array, items: { type: string } } featuredTestimonials: { type: array, items: { type: string } } responses: ‘200’: { description: OK } ‘401’: { description: Unauthorized }

———— Admin leads ———— /api/admin/leads/: get: tags: [Admin Leads] summary: List leads with filters security: [{ bearerAuth: [] }] parameters: - $ref: '#/components/parameters/PageParam' - $ref: '#/components/parameters/LimitParam' - { name: status, in: query, schema: { type: string, description: leadStatus } } - { name: source, in: query, schema: { type: string } } - { name: startDate, in: query, schema: { type: string } } - { name: endDate, in: query, schema: { type: string } } - { name: search, in: query, schema: { type: string } } responses: '200': { description: OK, content: { application/json: { schema: { $ref: '#/components/schemas/PagedResponse' } } } } /api/admin/leads/stats: get: tags: [Admin Leads] summary: Lead funnel stats security: [{ bearerAuth: [] }] parameters: - { name: startDate, in: query, schema: { type: string } } - { name: endDate, in: query, schema: { type: string } } - { name: source, in: query, schema: { type: string } } responses: '200': { description: OK, content: { application/json: { schema: { type: object } } } } /api/admin/leads/export: get: tags: [Admin Leads] summary: Export leads as CSV download security: [{ bearerAuth: [] }] parameters: - { name: startDate, in: query, schema: { type: string } } - { name: endDate, in: query, schema: { type: string } } - { name: source, in: query, schema: { type: string } } - { name: status, in: query, schema: { type: string } } responses: '200': { description: CSV file, content: { text/csv: { schema: { type: string } } } } } /api/admin/leads/{id}: get: tags: [Admin Leads] summary: One lead security: [{ bearerAuth: [] }] parameters: - $ref: '#/components/parameters/IdParam' responses: '200': { description: OK, content: { application/json: { schema: { $ref: '#/components/schemas/Lead' } } } } put: tags: [Admin Leads] summary: Update leadStatus/appointmentStatus/notes/treatmentInterest security: [{ bearerAuth: [] }] parameters: - $ref: '#/components/parameters/IdParam' requestBody: content: application/json: schema: type: object properties: leadStatus: { type: string, enum: [new, contacted, scheduled, completed, lost] } appointmentStatus: { type: string, enum: [none, booked, completed, cancelled, no_show] } notes: { type: string } treatmentInterest: { type: string } responses: '200': { description: OK, content: { application/json: { schema: { $ref: '#/components/schemas/Lead' } } } } delete: tags: [Admin Leads] summary: Delete lead security: [{ bearerAuth: [] }] parameters: - $ref: '#/components/parameters/IdParam' responses: '200': { description: OK }

———— Admin booking / redirects / analytics + dual mounts ———— /api/admin/booking-cta: get: tags: [Booking] summary: Same public booking-cta handler (no token needed) responses: '200': { description: OK } '404': { description: No active provider } /api/admin/leads/practo-click: post: tags: [Booking] summary: Same public click-log handler (always 200) responses: '200': { description: OK } /api/admin/booking-provider: get: tags: [Admin Misc] summary: Current booking provider config security: [{ bearerAuth: [] }] responses: '200': { description: OK } '404': { description: Not configured } put: tags: [Admin Misc] summary: Upsert booking provider config security: [{ bearerAuth: [] }] requestBody: content: application/json: schema: type: object properties: providerName: { type: string } profileUrl: { type: string } bookingUrl: { type: string } isActive: { type: boolean } responses: '200': { description: OK } /api/admin/practo-clicks: get: tags: [Admin Misc] summary: Practo click totals + 30-day daily stats security: [{ bearerAuth: [] }] parameters: - { name: startDate, in: query, schema: { type: string } } - { name: endDate, in: query, schema: { type: string } } responses: '200': { description: OK } /api/admin/redirects/: get: tags: [Admin Misc] summary: List redirects security: [{ bearerAuth: [] }] parameters: - $ref: '#/components/parameters/PageParam' - $ref: '#/components/parameters/LimitParam' - { name: search, in: query, schema: { type: string } } responses: '200': { description: OK, content: { application/json: { schema: { $ref: '#/components/schemas/PagedResponse' } } } } post: tags: [Admin Misc] summary: Create redirect (fromPath unique) security: [{ bearerAuth: [] }] requestBody: required: true content: application/json: schema: type: object required: [fromPath, toPath] properties: fromPath: { type: string } toPath: { type: string } statusCode: { type: integer, enum: [301, 302], default: 301 } responses: '201': {

description: Created } ‘409’: { description: Duplicate fromPath } /api/admin/redirects/{id}: get: tags: [Admin Misc] summary: One redirect security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ responses: ‘200’: { description: OK } ‘404’: { description: Not found } put: tags: [Admin Misc] summary: Update toPath/statusCode only (fromPath immutable) security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ requestBody: content: application/json: schema: type: object properties: toPath: { type: string } statusCode: { type: integer, enum: [301, 302] } responses: ‘200’: { description: OK } ‘404’: { description: Not found } delete: tags: [Admin Misc] summary: Delete redirect security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/IdParam’ responses: ‘200’: { description: OK } ‘404’: { description: Not found } /api/admin/analytics/reports/monthly: get: tags: [Admin Misc] summary: ‘Monthly analytics report (live: enquiry + click events)’ security: [{ bearerAuth: [] }] parameters: - { name: startDate, in: query, schema: { type: string } } - { name: endDate, in: query, schema: { type: string } } responses: ‘200’: { description: OK } /api/admin/analytics/events: get: tags: [Admin Misc] summary: Raw analytics event list security: [{ bearerAuth: [] }] parameters: - $ref: ‘#/components/parameters/PageParam’ - $ref: ‘#/components/parameters/LimitParam’ - { name: eventType, in: query, schema: { type: string } } - { name: startDate, in: query, schema: { type: string } } - { name: endDate, in: query, schema: { type: string } } responses: ‘200’: { description: OK, content: { application/json: { schema: { $ref: ‘#/components/schemas/PagedResponse’ } } } } }

`